B8.4 DVM messages

This section provides additional information on the various DVM messages, their associated fields, and flit packing. It contains the following subsections:

- B8.4.1 *DVM message payload*
- B8.4.2 *DVM message packing*
- B8.4.3 *TLB Invalidate*
- B8.4.4 *Branch Predictor Invalidate*
- B8.4.5 *Instruction Cache Invalidate*
- B8.4.6 *Synchronization*

B8.4.1 DVM message payload

The payload of a DVM operation from the Request Node to the Miscellaneous Node is distributed within:

- The Addr field in the DVM request from the Request Node.
- The lower 8 bytes of Data in the NonCopyBackWriteData packet.

The payload of a DVM operation from the Miscellaneous Node to the Request Node is distributed over two SnpDVMOp request packets using the Addr fields.

It is recommended, but not required, that the payload of a SnpDVMOp message matches the payload of the DVMOp message from which it originates.

Table B8.5 shows the various fields in the payload and their encodings.

Table B8.5: DVMOp fields and encodings

Field	Bits	Function
AddrV	1	Indicates if the message includes an address. `0b0` No address included `0b1` Address included
VIV	2	*Virtual Index* (VI) Valid. `0b00` VI not valid `0b01` Reserved `0b10` Reserved `0b11` VI is valid
VMIDV	1	`0b1` indicates that the *Virtual Machine Identifier* (VMID) is valid.
ASIDV	1	`0b1` indicates that the *Address Space Identifier* (ASID) is valid.
Security	2	Indicates which Security state the invalidation applies to. See Table B8.8 for the encodings for each DVMType.
Exception	2	Indicates that the transaction applies to: `0b00` Hypervisor and all Guest OS `0b01` EL3[a] `0b10` Guest OS `0b11` Hypervisor

Continued on next page

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.5 – *Continued from previous page*

Field	Bits	Function
DVMType	3	Indicates the DVM operation type as: `0b000` TLB Invalidate (TLBI) `0b001` Branch Predictor Invalidate (BPI) `0b010` Physical Instruction Cache Invalidate (PICI) `0b011` Virtual Instruction Cache Invalidate (VICI) `0b100` Synchronization `0b101-0b111` Reserved
VMID	8	Virtual Machine Identifier VMID[7:0]
ASID	16	Address Space Identifier
Stage	2	Indicates Stage 1 or Stage 1 invalidation: `0b00` DVMv7: Any transaction DVMv8: Both Stage 1 and Stage 2 invalidation `0b01` Stage 1 invalidation only `0b10` Stage 2 invalidation only `0b11` *Granule Protection Table* (GPT)
Leaf	1	Indicates whether only leaf entries are invalidated: `0b0` Invalidate all associated translations. `0b1` Invalidate Leaf Entry only.
Range[b]	1	Range can take any value. **When Range = 0** The transaction is a non-range based TLBI operation. Range is inapplicable and must be zero for non-TLBI DVM transactions. **When Range = 1** The transaction is a range-based TLBI operation.
Num[b]	5	Used as a constant multiplication factor in the range calculation. All binary values are valid.
Scale[b]	2	Used as a constant in address range exponent calculation. All binary values are valid.
TTL[b]	2	Hint of *Translation Table Level* (TTL) which includes the addresses to be invalidated. See Table B8.6 and Table B8.7 for details.

Continued on next page

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.5 – *Continued from previous page*

Field	Bits	Function
TG[b]	2	*Translation Granule* (TG) For non-range TLB Invalidations, TG and TTL indicate the table level hint, see Table B8.7. For TLB Invalidations by range, TG indicates the granule size: `0b00` Reserved `0b01` 4K `0b10` 16K `0b11` 64K
BaseAddr	37-41	Shifted base address of the range, shifted based on TG: **4K** BaseAddr is VA[MaxVA:12] **16K** BaseAddr is VA[MaxVA:14], VA[13:12] must be zero **64K** BaseAddr is VA[MaxVA:16], VA[15:12] must be zero
VA or	49-53	Virtual Address. This field is also used to transport the Intermediate Physical Address (IPA) for any TLBI by IPA operations.
PA	44-52	Physical Address. This field is used to transport the Physical Address for GPT TLBI by PA and PICI by PA operations.
VI	16	Virtual Index
VMIDExt	8	Virtual Machine Identifier VMID[15:8]

Continued on next page

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.5 – *Continued from previous page*

Field	Bits	Function
IS[c]	4	*Invalidation Size* (IS) encoding for GPT TLBI by PA operations, including Leaf only:
		`0b0000` 4K
		`0b0001` 16K
		`0b0010` 64K
		`0b0011` 2MB
		`0b0100` 32MB
		`0b0101` 512MB
		`0b0110` 1GB
		`0b0111` 16GB
		`0b1000` 64GB
		`0b1001` 512GB
		`0b1010-0b1111` Reserved

[a] DVMv8 only.

[b] Inapplicable and must be zero in DVM operations that are not TLBI.
See B8.4.3.1 *TLB Invalidate by Range* and B8.4.3.2 *Level Hint in TLBI operations* for details of how these fields are used in TLBI operations.

[c] Inapplicable and must be zero in DVM operations that are not GPT TLBI by PA or GPT TLBI by PA, Leaf only.
See B8.4.3.3 *Invalidation Size in GPT TLBI by PA operations*.

TTL and TG fields

For TLB Invalidations by address range, the TTL field can indicate which level of translation table walk holds the leaf entry for the address being invalidated. The encodings are shown in Table B8.6.

Table B8.6: Leaf entry hint for range-based TLB Invalidations

TTL	Meaning
`0b00`	No level hint information.
`0b01`	The leaf entry is on level 1 of the translation table walk.
`0b10`	The leaf entry is on level 2 of the translation table walk.
`0b11`	The leaf entry is on level 3 of the translation table walk.

For TLB Invalidations by non-range address, the TTL and TG fields indicate which level of translation table walk holds the leaf entry for the address being invalidated.

The encodings are shown in Table B8.7.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.7: Leaf entry hint for non-range TLB Invalidations

TG	TTL	Meaning
0b00	0b00	No level hint
	0b01	Reserved
	0b10	Reserved
	0b11	Reserved
0b01	0b00	The leaf entry is on level 0 of the translation table walk.
	0b01	The leaf entry is on level 1 of the translation table walk.
	0b10	The leaf entry is on level 2 of the translation table walk.
	0b11	The leaf entry is on level 3 of the translation table walk.
0b10	0b00	Reserved
	0b01	The leaf entry is on level 1 of the translation table walk.
	0b10	The leaf entry is on level 2 of the translation table walk.
	0b11	The leaf entry is on level 3 of the translation table walk.
0b11	0b00	Reserved
	0b01	The leaf entry is on level 1 of the translation table walk.
	0b10	The leaf entry is on level 2 of the translation table walk.
	0b11	The leaf entry is on level 3 of the translation table walk.

Security field

The Security field has different meanings depending on the DVMType, as shown in Table B8.8.

Table B8.8: Security field encodings for each DVMType

Security	TLBI	BPI	PICI		VICI
			Invalidation All	Invalidation by PA	
00	Realm	Secure and Non-secure	Root, Realm, Secure, and Non-secure	Root	Secure and Non-secure
01	Non-secure address from Secure context	Reserved	Realm and Non-secure	Realm	Reserved
10	Secure	Reserved	Secure and Non-secure	Secure	Secure
11	Non-secure	Reserved	Non-secure	Non-secure	Non-secure

ASID field

The ASID field contains an 8-bit or 16-bit Address Space Identifier.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

- Armv7 supports an 8-bit ASID.
- From Armv8, an 8-bit and 16-bit ASID are supported.

It cannot be determined from a DVM message whether the message uses an 8-bit or 16-bit ASID.

All 8-bit ASID messages are required to set the ASID[15:8] bits to zero.

It is expected that most systems will use a single ASID size across the entire system, either 8-bit ASID or 16-bit ASID.

In a system that contains a mix of 8-bit ASID and 16-bit ASID components, it is expected that all maintenance is done by an agent that uses 16-bit ASID. This ensures that the agent can perform maintenance on both the 8-bit ASID and 16-bit ASID components.

The interoperability requirements include:

- For an 8-bit ASID agent sending a message to a 16-bit ASID agent, a message appears as a 16-bit ASID with the upper 8 bits set to zero.
- For a 16-bit ASID agent sending a message to an 8-bit VMID agent:
  - If the upper 8 bits are zero, the message was received correctly.
  - If the upper 8 bits are non-zero, over-invalidation occurs as the 8-bit ASID agent ignores the upper 8 bits.

VMID field

The VMID field contains an 8-bit or 16-bit Virtual Machine Identifier.

- Armv7 and Armv8 support an 8-bit VMID.
- From Armv8.1, an 8-bit and 16-bit VMID are supported.

It cannot be determined from a DVM message whether the message uses an 8-bit or 16-bit VMID.

All 8-bit VMID messages are required to set the VMID[15:8] field to zero.

It is expected that most systems use a single VMID size across the entire system, either 8-bit VMID or 16-bit VMID.

In a system that contains both 8-bit VMID and 16-bit VMID components, it is expected that all maintenance is done by an agent that uses 16-bit VMID. This ensures that the agent can perform maintenance on both the 8-bit VMID and 16-bit VMID components.

The interoperability requirements include:

- For an 8-bit VMID agent sending a message to a 16-bit VMID agent, a message appears as a 16-bit VMID with the upper 8 bits set to zero.
- For a 16-bit VMID agent sending a message to an 8-bit VMID agent:
  - If the upper 8 bits are zero, the message was received correctly.
  - If the upper 8 bits are non-zero, over-invalidation occurs as the 8-bit VMID agent ignores the upper 8 bits.

From Armv8.1, VMIDExt is used to transport the upper byte of 16-bit VMIDs.

DVM domain

The SnpAttr bit in a DVM request is used to differentiate between Inner and Outer domain.

Table B8.9 shows the SnpAttr value encoding in DVM transactions.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.9: SnpAttr value encoding in DVM transactions

SnpAttr	Domain value
0	Inner domain
1	Outer domain

Two additional optional interface broadcast pins are defined, **BROADCASTTLBIINNER** and **BROADCASTTLBIOUTER**. They determine broadcasting of TLBI operations in the interconnect.

B8.4.2 DVM message packing

Table B8.10 shows the distribution of the payload in the DVMOp request from the Request Node, using 8-byte write semantics, and the distribution of the payload in the SnpDVMOp requests from the Miscellaneous Node.

In the DVMOp, the combination of the address field in the request and the 8-byte write data transports the complete payload. REQ.Addr[3] is not used in the request and must be zero.

In the two SnpDVMOp requests the combination of the two address fields transports the complete payload. SNP.Addr[0] is used in a SnpDVMOp request to indicate which part of the payload is being transported.

The valid combinations of Maximum PA (MPA) and Maximum VA (MVA) address bits are:

- MPA = 44 : MVA = 49
- MPA = 45 : MVA = 51
- MPA = 46 to 52 : MVA = 53

See B8.4.3.1 *TLB Invalidate by Range* and B8.4.3.2 *Level Hint in TLBI operations*.

Table B8.10: DVMOp and SnpDVMOp request payloads

X in REQ.Addr[x] DAT.Data[x]	Num bits	DVMOp REQ	DVMOp DAT
2:0	3	-	Num[2:0][a]
3	1	0	Num[3][a]
4	1	AddrV	Scale[0] PA[6] VA[6]
5	1	VMIDV VIV[0]	Scale[1] PA[7] VA[7]
6	1	ASIDV VIV[1]	IS[0] TTL[0] PA[8] VA[8]
7	1	Security[0]	IS[1] TTL[1] PA[9] VA[9]
8	1	Security[1]	IS[2] TG[0] PA[10] VA[10]

X in SNP.Addr[x]	Num bits	SnpDVMOp Request Part 1	SnpDVMOp Request Part 2
0	1	0	1
1	1	AddrV	Scale[0] PA[6] VA[6]
2	1	VMIDV VIV[0]	Scale[1] PA[7] VA[7]
3	1	ASIDV VIV[1]	IS[0] TTL[0] PA[8] VA[8]
4	1	Security[0]	IS[1] TTL[1] PA[9] VA[9]
5	1	Security[1]	IS[2] TG[0] PA[10] VA[10]

Continued on next page

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.10 – *Continued from previous page*

X in REQ.Addr[x] DAT.Data[x]	Num bits	DVMOp REQ	DVMOp DAT
9	1	Exception[0]	IS[3] TG[1] PA[11] VA[11]
10	1	Exception[1]	VA[12] PA[12]
13:11	3	DVMType[2:0]	VA[15:13] PA[15:13]
21:14	8	VMID[7:0] VI[27:20]	VA[23:16] PA[23:16]
37:22	16	ASID[15:0] VI[19:12][b]	VA[39:24] PA[39:24]
39:38	2	Stage	VA[41:40] PA[41:40]
40	1	Leaf	VA[42] PA[42]
41	1	Range[c]	VA[43] PA[43]
42	1	Num[4][a]	VA[44] PA[44]
43	1	-	VA[45] PA[45]
44	1	-	VA[46] PA[46]
45	1	-	VA[47] PA[47]
48:46	3	-	VA[50:48] PA[50:48]
49	1	-	VA[51] PA[51]
50	1	-	VA[52]
55:51	5	-	-
63:56	8	-	VMID[15:8][d]

X in SNP.Addr[x]	Num bits	SnpDVMOp Request Part 1	SnpDVMOp Request Part 2
6	1	Exception[0]	IS[3] TG[1] PA[11] VA[11]
7	1	Exception[1]	VA[12] PA[12]
10:8	3	DVMType[2:0]	VA[15:13] PA[15:13]
18:11	8	VMID[7:0] VI[27:20]	VA[23:16] PA[23:16]
34:19	16	ASID[15:0] VI[19:12][b]	VA[39:24] PA[39:24]
36:35	2	Stage	VA[41:40] PA[41:40]
37	1	Leaf	VA[42] PA[42]
38	1	VA[46]	VA[43] PA[43]
39	1	VA[47]	VA[44] PA[44]
40	1	VA[48]	VA[45] PA[45]
41	1	VA[50]	VA[49] PA[46]
42	1	VA[52]	VA[51] PA[47]
45:43	3	-	PA[50:48]
46	1	-	PA[51]
47	1	-	-
48	1	-	-

[a] For Part 2 of a SnpDVMOp request, Num[4:0] from the corresponding DVMOp request is carried on the FwdNID field of the Snoop flit alongside the Addr field. See Table B8.14.

[b] When used as Virtual Index VA (VI VA), REQ.Addr[37:30], DAT.Data[37:30], and SNP.Addr[34:27] can take any value.

[c] For Part 1 of a SnpDVMOp request, Range from the corresponding DVMOp request is carried on the FwdNID field of the Snoop flit alongside the Addr field. See Table B8.14.

[d] For Part 1 of a SnpDVMOp request, VMID[15:8] from the corresponding DVMOp request is carried on the VMIDExt field of the Snoop flit alongside the Addr field. See Table B13.8.

B8.4.3 TLB Invalidate

This section details the TLB Invalidate (TLBI) message.

For a TLBI message, some fields have a fixed value, as shown in Table B8.11

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.11: Fixed field values for a TLBI message

Field	Value	Status
DVMType	0b000	TLBI

The entries on which the TLBI must operate depends on the fields in the message.

Table B8.12 shows all the supported TLBI operations.

The Arm column indicates the minimum Arm architecture version required to support the message.

Table B8.12: TLBI messages

Operation	Arm	Exception	Security	VMIDV	ASIDV	Leaf	Stage	AddrV
EL3 TLBI all	v8	0b01	0b10	0b0	0b0	0b0	0b00	0b0
EL3 TLBI by VA	v8	0b01	0b10	0b0	0b0	0b0	0b00	0b1
EL3 TLBI by VA, Leaf only	v8	0b01	0b10	0b0	0b0	0b1	0b00	0b1
Secure Guest OS TLBI by Non-secure IPA	v8.4	0b10	0b01	0b1	0b0	0b0	0b10	0b1
Secure Guest OS TLBI by Non-secure IPA, Leaf only	v8.4	0b10	0b01	0b1	0b0	0b1	0b10	0b1
Secure TLBI all	v7	0b10	0b10	0b0	0b0	0b0	0b00	0b0
Secure TLBI by VA	v7	0b10	0b10	0b0	0b0	0b0	0b00	0b1
Secure TLBI by VA, Leaf only	v8	0b10	0b10	0b0	0b0	0b1	0b00	0b1
Secure TLBI by ASID	v7	0b10	0b10	0b0	0b1	0b0	0b00	0b0
Secure TLBI by ASID and VA	v7	0b10	0b10	0b0	0b1	0b0	0b00	0b1
Secure TLBI by ASID and VA, Leaf only	v8	0b10	0b10	0b0	0b1	0b1	0b00	0b1
Secure Guest OS TLBI all	v8.4	0b10	0b10	0b1	0b0	0b0	0b00	0b0
Secure Guest OS TLBI by VA	v8.4	0b10	0b10	0b1	0b0	0b0	0b00	0b1
Secure Guest OS TLBI all, Stage 1 only	v8.4	0b10	0b10	0b1	0b0	0b0	0b01	0b0
Secure Guest OS TLBI by Secure IPA	v8.4	0b10	0b10	0b1	0b0	0b0	0b10	0b1
Secure Guest OS TLBI by VA, Leaf only	v8.4	0b10	0b10	0b1	0b0	0b1	0b00	0b1
Secure Guest OS TLBI by Secure IPA, Leaf only	v8.4	0b10	0b10	0b1	0b0	0b1	0b10	0b1
Secure Guest OS TLBI by ASID	v8.4	0b10	0b10	0b1	0b1	0b0	0b00	0b0
Secure Guest OS TLBI by ASID and VA	v8.4	0b10	0b10	0b1	0b1	0b0	0b00	0b1
Secure Guest OS TLBI by ASID and VA, Leaf only	v8.4	0b10	0b10	0b1	0b1	0b1	0b00	0b1
All OS TLBI all	v7	0b10	0b11	0b0	0b0	0b0	0b00	0b0
Guest OS TLBI all, Stage 1 and 2	v7	0b10	0b11	0b1	0b0	0b0	0b00	0b0

Continued on next page

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.12 – *Continued from previous page*

Operation	Arm	Exception	Security	VMIDV	ASIDV	Leaf	Stage	AddrV
Guest OS TLBI by VA	v7	0b10	0b11	0b1	0b0	0b0	0b00	0b1
Guest OS TLBI all, Stage 1 only	v8	0b10	0b11	0b1	0b0	0b0	0b01	0b0
Guest OS TLBI by IPA	v8	0b10	0b11	0b1	0b0	0b0	0b10	0b1
Guest OS TLBI by VA, Leaf only	v8	0b10	0b11	0b1	0b0	0b1	0b00	0b1
Guest OS TLBI by IPA, Leaf only	v8	0b10	0b11	0b1	0b0	0b1	0b10	0b1
Guest OS TLBI by ASID	v7	0b10	0b11	0b1	0b1	0b0	0b00	0b0
Guest OS TLBI by ASID and VA	v7	0b10	0b11	0b1	0b1	0b0	0b00	0b1
Guest OS TLBI by ASID and VA, Leaf only	v8	0b10	0b11	0b1	0b1	0b1	0b00	0b1
Secure Hypervisor TLBI all	v8.4	0b11	0b10	0b0	0b0	0b0	0b00	0b0
Secure Hypervisor TLBI by VA	v8.4	0b11	0b10	0b0	0b0	0b0	0b00	0b1
Secure Hypervisor TLBI by VA, Leaf only	v8.4	0b11	0b10	0b0	0b0	0b1	0b00	0b1
Secure Hypervisor TLBI by ASID	v8.4	0b11	0b10	0b0	0b1	0b0	0b00	0b0
Secure Hypervisor TLBI by ASID and VA	v8.4	0b11	0b10	0b0	0b1	0b0	0b00	0b1
Secure Hypervisor TLBI by ASID and VA, Leaf only	v8.4	0b11	0b10	0b0	0b1	0b1	0b00	0b1
Hypervisor TLBI all	v7	0b11	0b11	0b0	0b0	0b0	0b00	0b0
Hypervisor TLBI by VA	v7	0b11	0b11	0b0	0b0	0b0	0b00	0b1
Hypervisor TLBI by VA, Leaf only	v8	0b11	0b11	0b0	0b0	0b1	0b00	0b1
Hypervisor TLBI by ASID	v8.1	0b11	0b11	0b0	0b1	0b0	0b00	0b0
Hypervisor TLBI by ASID and VA	v8.1	0b11	0b11	0b0	0b1	0b0	0b00	0b1
Hypervisor TLBI by ASID and VA, Leaf only	v8.1	0b11	0b11	0b0	0b1	0b1	0b00	0b1
Realm TLBI all	v9.2	0b10	0b00	0b0	0b0	0b0	0b00	0b0
Realm Guest OS TLBI all, Stage 1 only	v9.2	0b10	0b00	0b1	0b0	0b0	0b01	0b0
Realm Guest OS TLBI all, Stage 1 and 2	v9.2	0b10	0b00	0b1	0b0	0b0	0b00	0b0
Realm Guest OS TLBI by VA	v9.2	0b10	0b00	0b1	0b0	0b0	0b00	0b1
Realm Guest OS TLBI by VA, Leaf only	v9.2	0b10	0b00	0b1	0b0	0b1	0b00	0b1
Realm Guest OS TLBI by ASID	v9.2	0b10	0b00	0b1	0b1	0b0	0b00	0b0
Realm Guest OS TLBI by ASID and VA	v9.2	0b10	0b00	0b1	0b1	0b0	0b00	0b1
Realm Guest OS TLBI by ASID and VA, Leaf only	v9.2	0b10	0b00	0b1	0b1	0b1	0b00	0b1
Realm Guest OS TLBI by IPA	v9.2	0b10	0b00	0b1	0b0	0b0	0b10	0b1
Realm Guest OS TLBI by IPA, Leaf only	v9.2	0b10	0b00	0b1	0b0	0b1	0b10	0b1

Continued on next page

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.12 – *Continued from previous page*

Operation	Arm	Exception	Security	VMIDV	ASIDV	Leaf	Stage	AddrV
Realm Hypervisor TLBI all	v9.2	`0b11`	`0b00`	`0b0`	`0b0`	`0b0`	`0b00`	`0b0`
Realm Hypervisor TLBI by VA	v9.2	`0b11`	`0b00`	`0b0`	`0b0`	`0b0`	`0b00`	`0b1`
Realm Hypervisor TLBI by VA, Leaf only	v9.2	`0b11`	`0b00`	`0b0`	`0b0`	`0b1`	`0b00`	`0b1`
Realm Hypervisor TLBI by ASID	v9.2	`0b11`	`0b00`	`0b0`	`0b1`	`0b0`	`0b00`	`0b0`
Realm Hypervisor TLBI by ASID and VA	v9.2	`0b11`	`0b00`	`0b0`	`0b1`	`0b0`	`0b00`	`0b1`
Realm Hypervisor TLBI by ASID and VA, Leaf only	v9.2	`0b11`	`0b00`	`0b0`	`0b1`	`0b1`	`0b00`	`0b1`
GPT TLBI by PA	v9.2	`0b01`	`0b10`	`0b0`	`0b0`	`0b0`	`0b11`	`0b1`
GPT TLBI by PA, Leaf only	v9.2	`0b01`	`0b10`	`0b0`	`0b0`	`0b1`	`0b11`	`0b1`
GPT TLBI all	v9.2	`0b01`	`0b10`	`0b0`	`0b0`	`0b0`	`0b11`	`0b0`

B8.4.3.1 TLB Invalidate by Range

When DVM_Support is DVM_v8.4 or greater, TLBI operations by IPA or VA have the option to operate on an address range if the Range field is `0b1`.

Range-based TLBI operations include range-specific payload in addition to the payload fields in non-range based TLBI operation.

B8.4.3.1.1 Range-based payload packing for TLBI by VA and IPA operations

The range-based fields for TLBI operations by VA and IPA are:

- Range
- BaseAddr
- TG
- TTL
- Scale
- Num

Note

The TG and TTL fields can be used as Level hints for Non-range based TLBI operations by VA or IPA. See B8.4.3.2 *Level Hint in TLBI operations*.

When the Range field is set to 1 for TLBI operations by VA or IPA, the address range to invalidate is calculated using the following formula, where Translation_Granule_Size in bytes is determined from the TG value, as shown in Table B8.13:

$$BaseAddr \leq AddressRange < BaseAddr + \Big((Num + 1) \times 2^{(5 \times Scale + 1)} \times Translation_Granule_Size\Big)$$

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.13: TG field encodings in TLB instructions that apply to a range of addresses

TG	Translation granule size	Translation_Granule_Size
00	Reserved	Not applicable
01	4KB translation granule	4096
10	16KB translation granule	16384
11	64KB translation granule	65536

The range parameters are placed in the request packets as shown in Table B8.10.

The Range and Num values are carried on the FwdNID field in the Snoop flit alongside the Addr field for the corresponding Snoop transaction. Unused bits of FwdNID in SnpDVMOp must be zero.

Table B8.14 shows the placement of the Range and Num values in the SnpDVMOp payload.

Table B8.14: Num value placement in SnpDVMOp payload

FwdNID bit	SnpDVMOp request	
	Part 1	Part 2
0	Range	Num[0]
1	0	Num[1]
2	0	Num[2]
3	0	Num[3]
4	0	Num[4]
5	0	0
6	0	0

B8.4.3.2 Level Hint in TLBI operations

Non-range based TLBI operations by VA or IPA operations are permitted to use TG and TTL fields. These operations use TG and TTL as an indication of the level of the page table walk that holds the leaf entry for the address that is being invalidated. For such operations, the following conditions apply:

- The Range field must be zero.
- The Num and Scale fields are inapplicable and must be zero.

B8.4.3.3 Invalidation Size in GPT TLBI by PA operations

GPT TLBI by PA operations perform range-based invalidation and invalidate TLB entries starting from the PA, within the range as specified in the IS field.

The range-based fields for GPT TLBI operations by PA are IS and PA. If the PA is not aligned to the IS value, no TLB entries are required to be invalidated.

The IS field is applicable only in GPT TLBI by PA operations.

The Range field is applicable and must be 1 for GPT TLBI by PA operations.

The Range field is applicable and must be 0 for GPT TLBI all operations.

The Num and Scale fields are inapplicable and must be 0 for GPT TLBI operations.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.4.4 Branch Predictor Invalidate

This section describes the *Branch Predictor Invalidate* (BPI) operations.

The BPI message is used to invalidate virtual addresses from branch predictors.

The fixed field values for a BPI message are shown in Table B8.15.

Table B8.15: Fixed field values for a Branch Predictor Invalidate message

Field	Value	Status
DVMType	`0b001`	Branch Predictor Invalidate
Part Num	-	See Table B8.10
VMIDV	`0b0`	VMID field not valid
ASIDV	`0b0`	ASID field not valid
Security	`0b00`	Applies to both Secure and Non-secure
Exception	`0b00`	Applies to all Guest OS and Hypervisor
VMID VMIDExt	`0xXX`	VMID not specified
ASID	`0xXXXX`	ASID not specified
Stage	`0b00`	Reserved, set to 0
Leaf	`0b0`	Reserved, set to 0

Note

The use of BPI with a 16-bit ASID is not supported.

Table B8.16 shows the operations supported by BPI.

Table B8.16: Branch Predictor Invalidate operations

Operation	Arm	AddrV
Branch Predictor Invalidate all	v7	`0b0`
Branch Predictor Invalidate by VA	v7	`0b1`

B8.4.5 Instruction Cache Invalidate

Instruction caches can use either a PA or a VA to tag the data they contain. A system could contain a mixture of both forms of cache.

The DVM protocol includes instruction cache invalidation operations that use Physical Addresses and operations that use Virtual Addresses.

A component that receives DVM messages must support both forms of message, independent of the style of instruction cache implemented. In an instance where a message is received in a format that is not native to the cache type, over-invalidation might be required.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

B8.4.5.1 Physical Instruction Cache Invalidate

This section describes the *Physical Instruction Cache Invalidate* (PICI) operations that the DVM message supports. This message type is also used for Instruction Caches which are *Virtually Indexed Physically Tagged* (VIPT).

The fixed field values for a PICI message are shown in Table B8.17.

Table B8.17: Fixed field values for a Physical Instruction Cache Invalidate message

Field	Value	Status
DVMType	`0b010`	Physical Instruction Cache Invalidate
Part Num	-	See Table B8.10
Exception	`0b00`	Applies to all Guest OS and Hypervisor
Stage	`0b00`	Reserved, set to 0
Leaf	`0b0`	Reserved, set to 0

All supported PICI operations are shown in Table B8.18.

Table B8.18: Physical Instruction Cache Invalidate operations

Operation	Arm	Security	VIV	AddrV
PICI all Root, Realm, Secure, and Non-secure	v9.2	`0b00`	`0b00`	`0b0`
PICI by PA without Virtual Index, Root only	v9.2	`0b00`	`0b00`	`0b1`
PICI by PA with Virtual Index, Root only	v9.2	`0b00`	`0b11`	`0b1`
PICI all Realm and Non-secure	v9.2	`0b01`	`0b00`	`0b0`
PICI by PA without Virtual Index, Realm only	v9.2	`0b01`	`0b00`	`0b1`
PICI by PA with Virtual Index, Realm only	v9.2	`0b01`	`0b11`	`0b1`
PICI all Secure and Non-secure	v7	`0b10`	`0b00`	`0b0`
PICI by PA without Virtual Index, Secure only	v7	`0b10`	`0b00`	`0b1`
PICI by PA with Virtual Index, Secure only	v7	`0b10`	`0b11`	`0b1`
PICI all, Non-secure only	v7	`0b11`	`0b00`	`0b0`
PICI by PA without Virtual Index, Non-secure only	v7	`0b11`	`0b00`	`0b1`
PICI by PA with Virtual Index, Non-secure only	v7	`0b11`	`0b11`	`0b1`

Note

When VIV is `0b11`, VI[19:12] is used as part of the PA.

B8.4.5.2 Virtual Instruction Cache Invalidate

This section describes the *Virtual Instruction Cache Invalidate* (VICI) operations.

The fixed field values for a VICI message are shown in Table B8.19.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.19: Fixed field values for a Virtual Instruction Cache Invalidate message

Field	Value	Status
DVMType	`0b011`	Virtual Instruction Cache Invalidate
Part Num	-	See Table B8.10
Stage	`0b00`	Reserved, set to 0
Leaf	`0b0`	Reserved, set to 0

Table B8.20 shows the operations supported by VICI.

Table B8.20: Virtual Instruction Cache Invalidate operations

Operation	Arm	Exception	Security	VMIDV	ASIDV	AddV
Hypervisor and all Guest OS VICI all, Secure and Non-secure	v7	`0b00`	`0b00`	`0b0`	`0b0`	`0b0`
Hypervisor and all Guest OS VICI all, Non-secure only	v7	`0b00`	`0b11`	`0b0`	`0b0`	`0b0`
All Guest OS VICI by ASID and VA, Secure only	v7	`0b10`	`0b10`	`0b0`	`0b1`	`0b1`
All Guest OS VICI by VMID, Secure only	v8.4	`0b10`	`0b10`	`0b1`	`0b0`	`0b0`
All Guest OS VICI by ASID, VA and VMID, Secure only	v8.4	`0b10`	`0b10`	`0b1`	`0b1`	`0b1`
All Guest OS VICI by VMID, Non-secure only	v7	`0b10`	`0b11`	`0b1`	`0b0`	`0b0`
All Guest OS VICI by ASID, VA and VMID, Non-secure only	v7	`0b10`	`0b11`	`0b1`	`0b1`	`0b1`
Hypervisor VICI by VA, Non-secure only	v7	`0b11`	`0b11`	`0b0`	`0b0`	`0b1`
Hypervisor VICI by ASID and VA, Non-secure only	v8.1	`0b11`	`0b11`	`0b0`	`0b1`	`0b1`

B8.4.6 Synchronization

This section describes the DVMSync operation.

A *Synchronization* (Sync) message is used when the Requester needs to know when all previous invalidations are complete.

The fixed field values for a Sync message are shown in Table B8.21.

Table B8.21: Sync operation fixed values

Field	Value	Status
DVMType	`0b100`	Synchronization message.
Part Num	-	See Table B8.10.
AddrV	`0b0`	No address information is available.
VMIDV	`0b0`	No VMID information is available.

Continued on next page

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Table B8.21 – *Continued from previous page*

Field	Value	Status
ASIDV	`0b0`	No ASID information is available.
Security	`0b00`	Security information is not applicable.
Exception	`0b00`	Exception information is not applicable.
VMID VMIDExt	`0xXX`	VMID is not specified.
ASID	`0xXXXX`	ASID is not specified.
Stage	`0b00`	Stage information is not applicable.
Leaf	`0b0`	Leaf information is not applicable.

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential

Chapter B9
Error Handling

This chapter describes the error handling requirements. It contains the following sections:

- B9.1 *Packet level*
- B9.2 *Sub-packet level*
- B9.3 *Use of interface parity*
- B9.4 *Hardware and software error categories*

Copyright © 2014, 2017-2025 Arm Limited or its affiliates. All rights reserved.
Non-confidential